Phase 2 — Python & Programming

ExamGuard AI — Implementation Guide

Python Basics

What is Python?

Python is a programming language - a way to write instructions that a computer can follow. It's the language you'll use to build ExamGuard and virtually every AI/ML project in the world.

Python is popular for AI because:

- It's readable (looks almost like English)
- It has thousands of ready-made libraries for AI, data, and math
- It's the standard language used by Google, Meta, OpenAI, and every major AI lab

Why Python Matters for ExamGuard

Every single line of ExamGuard will be written in Python.

- YOLO object detection? Python.
- CNN for gaze tracking? Python.
- LSTM for behavior analysis? Python.
- Autoencoder for anomaly detection? Python.
- RL agent for alert decisions? Python.
- Dashboard? Python (with a web framework).
- Data processing? Python.
- Model training? Python.

There is no ExamGuard without Python. It's not optional - it's the foundation of everything.

What to Learn

1. Variables (Storing Information)

Variables hold data that your program uses.

```
# ExamGuard examples:
student_count = 100
camera_count = 5
confidence_threshold = 0.75
is_cheating = False
student_name = "Ahmed"
```

ExamGuard connection: You'll store detection results, confidence scores, camera IDs, and alert statuses in variables.

2. Data Types

Type	What It Stores	Example	ExamGuard Use

`int`	Whole numbers	`camera_id = 3`	Camera numbers, student counts
`float`	Decimal numbers	`confidence = 0.87`	Model confidence scores
`str`	Text	`alert = "Phone detected"`	Alert messages
`bool`	True/False	`is_suspicious = True`	Decision flags

3. Lists (Collections of Items)

```
# List of camera feeds
cameras = ["cam_1", "cam_2", "cam_3", "cam_4", "cam_5"]

# List of detection scores
scores = [0.45, 0.87, 0.23, 0.91, 0.12]

# Access items
first_camera = cameras[0]    # "cam_1"
highest_score = max(scores)  # 0.91
```

ExamGuard connection: Store lists of camera feeds, detection results, alert histories.

4. Dictionaries (Key-Value Pairs)

```
# An alert as a dictionary
alert = {
   "student_location": "Row 3, Seat 7",
   "behavior": "Phone detected",
   "confidence": 0.89,
   "camera": "cam_2",
   "timestamp": "10:23:45"
}

# Access values
print(alert["behavior"])    # "Phone detected"
print(alert["confidence"])  # 0.89
```

ExamGuard connection: Every alert, every detection result, every student record will be stored as a dictionary.

5. If/Else (Making Decisions)

```
confidence = 0.87
threshold = 0.75

if confidence >= threshold:
   print("ALERT: Suspicious behavior detected!")
   send_alert_to_invigilator()
elif confidence >= 0.50:
   print("WATCH: Monitoring this student closely")
   increase_camera_fps()
else:
   print("NORMAL: No action needed")
```

ExamGuard connection: The entire alert system is based on if/else decisions - when to alert, what severity, when to ignore.

6. Loops (Repeating Actions)

```
# Process frames from all cameras
cameras = ["cam_1", "cam_2", "cam_3", "cam_4", "cam_5"]

for camera in cameras:
    frame = get_frame(camera)
    detections = run_yolo(frame)
    if detections:
        analyze_behavior(detections)
```

ExamGuard connection: You'll loop through camera feeds, loop through video frames, loop through detection results, loop through training data.

7. Functions (Reusable Blocks of Code)

```
def check_for_phone(frame):
    """Run YOLO on a frame to detect phones."""
    detections = yolo_model.predict(frame)
    phones = [d for d in detections if d.label == "phone"]
    return phones

def should_alert(confidence, threshold=0.75):
    """Decide whether to send an alert."""
    return confidence >= threshold

# Use the functions
frame = get_camera_frame("cam_1")
phones = check_for_phone(frame)
if phones and should_alert(phones[0].confidence):
    send_alert("Phone detected!")
```

ExamGuard connection: You'll write functions for every step: processing frames, running models, making decisions, sending alerts.

8. Classes (Organizing Complex Code)

```
class ExamCamera:
    def __init__(self, camera_id, location):
        self.camera_id = camera_id
        self.location = location
        self.fps = 10  # default fps
        self.is_active = True

    def get_frame(self):
        """Capture a frame from this camera."""
        # ... capture logic ...
        pass

    def increase_fps(self):
        """Switch to high FPS mode for suspicious activity."""
        self.fps = 30

# Create camera objects
```

```
cam1 = ExamCamera("cam_1", "Front-Left")
cam2 = ExamCamera("cam_2", "Front-Center")
```

ExamGuard connection: Each camera, each model, each alert will be organized as a class. This keeps the code clean and manageable.

Mini Project: Exam Score Classifier

Build this simple program to practice ALL the basics:

```
"""
Mini Project: Exam Score Classifier
Practice: variables, input, if/else, functions, lists, loops
"""

def classify_score(score):
    """Classify a score as pass, fail, or distinction."""
    if score >= 80:
        return "Distinction"
    elif score >= 50:
        return "Pass"
    else:
        return "Fail"

def calculate_statistics(scores):
    """Calculate basic stats for a list of scores."""
    average = sum(scores) / len(scores)
    highest = max(scores)
    lowest = min(scores)
    return average, highest, lowest

# Main program
scores = []

print("=== Exam Score Classifier ===")
print("Enter scores (type 'done' to finish):")

while True:
    user_input = input("Score: ")
    if user_input.lower() == "done":
        break
    score = int(user_input)
    result = classify_score(score)
    scores.append(score)
    print(f"  → {score}: {result}")

if scores:
    avg, high, low = calculate_statistics(scores)
    print(f"\n--- Statistics ---")
    print(f"Students: {len(scores)}")
    print(f"Average: {avg:.1f}")
    print(f"Highest: {high}")
    print(f"Lowest: {low}")
```

```
# Count results
distinctions = sum(1 for s in scores if s >= 80)
passes = sum(1 for s in scores if 50 <= s < 80)
fails = sum(1 for s in scores if s < 50)
print(f"Distinctions: {distinctions}")
print(f"Passes: {passes}")
print(f"Fails: {fails}")
```

Why this mini project?

It practices the SAME patterns you'll use in ExamGuard:

- Variables → storing scores = storing detection confidence
- Functions → classify_score = classify behavior
- If/else → score thresholds = confidence thresholds
- Lists → list of scores = list of detections
- Loops → process each score = process each frame
- Statistics → analyze scores = analyze model performance

Key Python Concepts for ExamGuard

Python Concept	ExamGuard Use
Variables	Store confidence scores, camera IDs, thresholds
Lists	Collections of frames, detections, alerts
Dictionaries	Alert data, student records, model configurations
If/else	Alert decisions, severity levels
For loops	Process each frame, each camera, each detection
While loops	Continuous monitoring loop
Functions	Each processing step is a function
Classes	Camera objects, model wrappers, alert managers
File I/O	Save/load model weights, log alerts
Error handling	What if a camera disconnects? Handle it gracefully

How Much Python Do You Need?

To START ExamGuard: Basic Python (variables, loops, functions) ~2-3 weeks
To BUILD models: Intermediate Python (classes, libraries) ~2-3 weeks
To COMPLETE ExamGuard: Comfortable Python (debugging, optimization) Ongoing

You don't need to be a Python expert to start. Learn the basics, then learn more as you need it. The best way to learn Python is by USING it on real projects.

NumPy - Numerical Python

What is NumPy?

NumPy (Numerical Python) is a Python library for working with arrays of numbers. Think of it as a super-powered calculator that can handle millions of numbers at once.

In simple terms: NumPy lets you do math on big collections of numbers very fast.

```
import numpy as np

# A regular Python list (slow)
numbers = [1, 2, 3, 4, 5]

# A NumPy array (fast!)
numbers = np.array([1, 2, 3, 4, 5])
```

Why NumPy Matters for ExamGuard

Images ARE NumPy Arrays

This is the key insight: every image is just a grid of numbers, and NumPy is how Python handles grids of numbers.

```
A camera frame (1920 x 1080 pixels, color) is stored as:

NumPy array with shape: (1080, 1920, 3)
                        |     |     |
                        |     |     └── 3 color channels (Red, Green, Blue)
                        |     └──────── 1920 pixels wide
                        └────────────── 1080 pixels tall

Total numbers: 1080 x 1920 x 3 = 6,220,800 numbers PER FRAME
```

Every pixel has 3 values (Red, Green, Blue), each between 0 and 255.

A single camera at 30 fps generates 186 million numbers per second. NumPy handles this efficiently.

Where NumPy is used in ExamGuard:

ExamGuard Task	NumPy's Role
Reading camera frames	Each frame = NumPy array
Resizing images	Reshape arrays for model input
Normalizing pixel values	Divide all values by 255 (scale 0-1)
Model input/output	All model predictions are NumPy arrays
Calculating distances	Person Re-ID vector comparisons
Statistics	Mean confidence, detection counts
Batch processing	Stack multiple frames into one array

What to Learn

1. Creating Arrays

```
import numpy as np

# From a list
scores = np.array([0.87, 0.45, 0.92, 0.31, 0.78])
```

```
# Create zeros (like a blank image)
blank_frame = np.zeros((1080, 1920, 3))   # Black image

# Create ones
ones = np.ones((224, 224, 3))           # White image

# Random values (useful for testing)
random_frame = np.random.randint(0, 255, (1080, 1920, 3))  # Random "image"

print(scores.shape)      # (5,)
print(blank_frame.shape)  # (1080, 1920, 3)
```

2. Array Properties

```
frame = np.random.randint(0, 255, (1080, 1920, 3), dtype=np.uint8)

print(frame.shape)    # (1080, 1920, 3) - dimensions
print(frame.dtype)    # uint8 - data type (0-255)
print(frame.size)     # 6220800 - total elements
print(frame.ndim)     # 3 - number of dimensions
print(frame.min())    # 0 - minimum value
print(frame.max())    # 255 - maximum value
print(frame.mean())   # ~127 - average value
```

3. Reshaping (Critical for Models!)

```
# YOLO needs input as (640, 640) but camera gives (1080, 1920)
# We need to resize!

# Original frame
frame = np.random.randint(0, 255, (1080, 1920, 3))
print(frame.shape)  # (1080, 1920, 3)

# Note: For actual image resizing, you'll use OpenCV (cv2.resize)
# But understanding reshape is fundamental

# Reshape a 1D array to 2D
flat = np.array([1, 2, 3, 4, 5, 6])
matrix = flat.reshape(2, 3)
print(matrix)
# [[1, 2, 3],
#  [4, 5, 6]]

# Flatten a 2D array to 1D (needed for some models)
flat_again = matrix.flatten()
print(flat_again)  # [1, 2, 3, 4, 5, 6]
```

4. Math Operations

```
# Normalize pixel values (REQUIRED before feeding to neural networks)
frame = np.random.randint(0, 255, (224, 224, 3), dtype=np.uint8)

# Pixel values: 0-255 → 0.0-1.0
```

```
normalized = frame / 255.0
print(normalized.max())   # 1.0
print(normalized.min())   # 0.0

# This is called NORMALIZATION - neural networks work better with 0-1 values

# Element-wise operations (happens to ALL numbers at once)
a = np.array([0.87, 0.45, 0.92, 0.31])
b = np.array([0.75, 0.50, 0.80, 0.60])

# Which detections are above threshold?
above_threshold = a > 0.5
print(above_threshold)  # [True, False, True, False]

# Average confidence
print(np.mean(a))      # 0.6375
```

5. Slicing (Extracting Parts)

```
# Get a region of an image (like cropping a face)
frame = np.random.randint(0, 255, (1080, 1920, 3))

# Crop region: rows 200-400, columns 500-700
face_crop = frame[200:400, 500:700, :]
print(face_crop.shape)  # (200, 200, 3) - a 200x200 face crop

# Get just the red channel
red_channel = frame[:, :, 0]
print(red_channel.shape)  # (1080, 1920)

# Get the top half of the image
top_half = frame[:540, :, :]
print(top_half.shape)  # (540, 1920, 3)
```

6. Stacking (Combining Arrays)

```
# Stack multiple frames into a batch (for GPU processing)
frame1 = np.random.rand(224, 224, 3)
frame2 = np.random.rand(224, 224, 3)
frame3 = np.random.rand(224, 224, 3)

# Create a batch of 3 frames
batch = np.stack([frame1, frame2, frame3])
print(batch.shape)  # (3, 224, 224, 3)
#                   |
#                   └── 3 frames in the batch

# Models process batches faster than individual frames!
```

7. Distance Calculations (for Person Re-ID)

```
# Person Re-ID: Compare two appearance vectors
person_cam1 = np.array([0.23, 0.87, 0.12, 0.45, 0.91])
person_cam3 = np.array([0.25, 0.85, 0.14, 0.43, 0.89])
```

```
different_person = np.array([0.91, 0.13, 0.78, 0.22, 0.34])

# Euclidean distance
dist_same = np.sqrt(np.sum((person_cam1 - person_cam3) ** 2))
dist_diff = np.sqrt(np.sum((person_cam1 - different_person) ** 2))

print(f"Same person distance: {dist_same:.4f}")     # ~0.04 (small = same person)
print(f"Different person distance: {dist_diff:.4f}")  # ~1.12 (large = different person)
```

Mini Project: Fake Image Processor

Build this to practice NumPy with image-like data:

```
"""
Mini Project: Fake Image Processor
Practice: NumPy arrays, reshaping, math operations, slicing
"""
import numpy as np

# Step 1: Create a fake "camera frame" (100x100 pixels, 3 colors)
print("=== ExamGuard Frame Simulator ===\n")

frame = np.random.randint(0, 255, (100, 100, 3), dtype=np.uint8)
print(f"Frame shape: {frame.shape}")
print(f"Frame size: {frame.size} numbers")
print(f"Pixel range: {frame.min()} to {frame.max()}")

# Step 2: Normalize (what we do before feeding to AI model)
normalized = frame / 255.0
print(f"\nNormalized range: {normalized.min():.2f} to {normalized.max():.2f}")

# Step 3: Crop a "face region" (rows 20-60, columns 30-70)
face = frame[20:60, 30:70, :]
print(f"Face crop shape: {face.shape}")

# Step 4: Simulate 5 camera frames in a batch
frames = np.random.randint(0, 255, (5, 100, 100, 3), dtype=np.uint8)
print(f"\nBatch of 5 frames shape: {frames.shape}")

# Step 5: Calculate average brightness per frame
for i in range(5):
    brightness = frames[i].mean()
    status = "BRIGHT" if brightness > 127 else "DARK"
    print(f"Camera {i+1}: avg brightness = {brightness:.1f} ({status})")

# Step 6: Simulate detection confidence scores
confidences = np.array([0.87, 0.45, 0.92, 0.31, 0.78, 0.65, 0.23, 0.91])
threshold = 0.70

alerts = confidences[confidences >= threshold]
print(f"\n{len(alerts)} detections above threshold {threshold}: {alerts}")
print(f"Average confidence of alerts: {alerts.mean():.2f}")
```

```
# Step 7: Simulate Person Re-ID
print("\n=== Person Re-ID Simulation ===")
person_a_cam1 = np.random.rand(128)  # 128-dim feature vector
person_a_cam3 = person_a_cam1 + np.random.normal(0, 0.02, 128)  # Same person, small noise
person_b_cam2 = np.random.rand(128)  # Different person

dist_same = np.linalg.norm(person_a_cam1 - person_a_cam3)
dist_diff = np.linalg.norm(person_a_cam1 - person_b_cam2)

print(f"Same person (Cam1 vs Cam3): distance = {dist_same:.4f}")
print(f"Different person (A vs B):  distance = {dist_diff:.4f}")
print(f"Match: {'YES' if dist_same < 0.5 else 'NO'} (threshold: 0.5)")
```

What this mini project teaches:

Step	NumPy Skill	ExamGuard Connection
Create frame	Array creation	Camera captures frame
Normalize	Division operation	Preprocess for model
Crop face	Array slicing	Extract face for gaze CNN
Batch frames	Array stacking	GPU batch processing
Brightness	Statistical operations	Image quality check
Threshold	Boolean indexing	Filter confident detections
Re-ID	Distance calculation	Match persons across cameras

Key NumPy Functions for ExamGuard

```
# Creation
np.array()          # Create from list
np.zeros()          # Create empty (black image)
np.ones()           # Create filled
np.random.randint()    # Random integers (fake images)
np.random.rand()       # Random floats 0-1

# Properties
array.shape          # Dimensions
array.dtype          # Data type
array.mean()         # Average
array.max(), .min()    # Range

# Operations
array / 255.0        # Normalize
array.reshape()       # Change dimensions
array.flatten()       # Make 1D
np.stack()          # Combine arrays
np.linalg.norm()       # Distance calculation

# Indexing
array[start:end]       # Slice
array[array > 0.5]     # Boolean filter
```

Common Gotcha for Beginners

```
# WRONG: Regular Python list math
a = [1, 2, 3]
b = [4, 5, 6]
c = a + b        # [1, 2, 3, 4, 5, 6]  ← concatenation, NOT addition!

# RIGHT: NumPy array math
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
c = a + b        # [5, 7, 9]  ← actual element-wise addition!
```

NumPy arrays do MATH. Python lists do CONCATENATION. This trips up every beginner at least once.

Pandas - Data Tables in Python

What is Pandas?

Pandas is a Python library for working with data in tables (rows and columns). Think of it as Excel, but inside Python, and much more powerful.

```
import pandas as pd

# This is a DataFrame - Pandas' main data structure
data = pd.DataFrame({
   "student_id": [1, 2, 3, 4, 5],
   "score": [85, 72, 91, 65, 78],
   "passed": [True, True, True, True, True]
})

print(data)
#   student_id  score  passed
# 0         1     85    True
# 1         2     72    True
# 2         3     91    True
# 3         4     65    True
# 4         5     78    True
```

Why Pandas Matters for ExamGuard

Before you train ANY model, you need to understand your data. Pandas is how you load, explore, clean, and prepare data.

ExamGuard Data Example

Imagine you have 10,000 labeled video clips for training. The metadata is stored in a CSV file:

```
clip_id,filename,label,duration_sec,camera_id,hall_id,student_position,confidence
1,clip_0001.mp4,normal,30,cam_1,hall_A,row2_seat5,0.95
2,clip_0002.mp4,phone_detected,30,cam_2,hall_A,row3_seat7,0.88
3,clip_0003.mp4,normal,30,cam_1,hall_A,row1_seat3,0.97
4,clip_0004.mp4,looking_at_neighbor,30,cam_3,hall_B,row4_seat2,0.72
5,clip_0005.mp4,normal,30,cam_2,hall_B,row2_seat8,0.93
...
```

Pandas lets you ask questions like:

- How many cheating clips vs normal clips? (imbalance check)
- Which camera catches the most suspicious behavior?
- What's the average confidence score per label?
- Are there any missing or corrupted entries?

What to Learn

1. Reading Data (Loading a CSV file)

```
import pandas as pd

# Load the dataset
```

```
df = pd.read_csv("exam_clips_metadata.csv")

# Quick look at the data
print(df.head())        # First 5 rows
print(df.shape)         # (10000, 8) → 10,000 rows, 8 columns
print(df.info())        # Data types and missing values
print(df.describe())    # Statistics for numeric columns
```

ExamGuard connection: Before training any model, you MUST load and understand your dataset.

2. Exploring Data

```
# How many clips per label?
print(df["label"].value_counts())
# normal            9500
# phone_detected     200
# looking_at_neighbor  180
# passing_notes       70
# other_suspicious    50

# PROBLEM! 9500 normal vs 500 total cheating = extremely imbalanced!
# This is the imbalanced data problem we discussed.
```

ExamGuard connection: This is how you discover the class imbalance problem BEFORE it ruins your model.

3. Filtering Rows

```
# Get only cheating clips
cheating = df[df["label"] != "normal"]
print(f"Cheating clips: {len(cheating)}")

# Get high-confidence detections only
high_conf = df[df["confidence"] >= 0.80]
print(f"High confidence: {len(high_conf)}")

# Get clips from a specific camera
cam2_clips = df[df["camera_id"] == "cam_2"]
print(f"Camera 2 clips: {len(cam2_clips)}")

# Combine filters: cheating + high confidence
reliable_cheating = df[(df["label"] != "normal") & (df["confidence"] >= 0.80)]
print(f"Reliable cheating clips: {len(reliable_cheating)}")
```

ExamGuard connection: Filter your training data to find exactly what you need for each model.

4. Group By (Analyzing Patterns)

```
# Average confidence per label
print(df.groupby("label")["confidence"].mean())
# label
# looking_at_neighbor  0.71
# normal               0.94
# other_suspicious     0.65
# passing_notes        0.68
# phone_detected       0.85
```

```
# Count clips per camera per hall
print(df.groupby(["hall_id", "camera_id"])["clip_id"].count())

# Which hall has the most suspicious activity?
suspicious = df[df["label"] != "normal"]
print(suspicious.groupby("hall_id")["clip_id"].count())
```

ExamGuard connection: Find patterns in your data. Which cameras, halls, or positions have the most cheating? This helps you understand the data before training.

5. Adding New Columns

```
# Add a binary label column (for training)
df["is_cheating"] = (df["label"] != "normal").astype(int)
# 0 = normal, 1 = cheating

# Add a duration category
df["duration_cat"] = pd.cut(df["duration_sec"], bins=[0, 15, 30, 60],
                            labels=["short", "medium", "long"])

# Add a row-column position
df["row"] = df["student_position"].str.extract(r"row(\d+)")
```

ExamGuard connection: Create the exact labels and features your model needs from raw data.

6. Handling Missing Data

```
# Check for missing values
print(df.isnull().sum())
# clip_id          0
# filename         0
# label            3   ← 3 clips have no label!
# duration_sec     0
# camera_id        1   ← 1 clip has no camera info
# confidence      12   ← 12 clips have no confidence score

# Drop rows with missing labels (can't train without labels!)
df_clean = df.dropna(subset=["label"])

# Fill missing confidence with average
df_clean["confidence"] = df_clean["confidence"].fillna(df_clean["confidence"].mean())

print(f"Clean dataset: {len(df_clean)} clips")
```

ExamGuard connection: Real data is always messy. Missing labels, corrupted files, incomplete records. Clean BEFORE training.

7. Saving Processed Data

```
# Save clean training data
train_data = df_clean[df_clean["hall_id"].isin(["hall_A", "hall_B", "hall_C"])]
test_data = df_clean[df_clean["hall_id"].isin(["hall_D", "hall_E"])]

train_data.to_csv("train_metadata.csv", index=False)
test_data.to_csv("test_metadata.csv", index=False)
```

```
print(f"Training clips: {len(train_data)}")
print(f"Testing clips: {len(test_data)}")
```

ExamGuard connection: Split data by hall (not randomly) so the test set has completely new environments.

Mini Project: Analyze ExamGuard Training Data

```
"""
Mini Project: Analyze ExamGuard Training Data
Practice: Reading CSV, filtering, groupby, statistics, missing data
"""
import pandas as pd

# Step 1: Create a sample dataset (since we don't have real data yet)
import numpy as np

np.random.seed(42)
n = 1000

data = {
    "clip_id": range(1, n + 1),
    "label": np.random.choice(
        ["normal", "phone", "looking_neighbor", "passing_notes", "suspicious"],
        size=n,
        p=[0.90, 0.04, 0.03, 0.02, 0.01]  # Imbalanced!
    ),
    "camera_id": np.random.choice(["cam_1", "cam_2", "cam_3", "cam_4", "cam_5"], size=n),
    "hall_id": np.random.choice(["hall_A", "hall_B", "hall_C"], size=n),
    "duration_sec": np.random.randint(10, 60, size=n),
    "confidence": np.round(np.random.uniform(0.3, 1.0, size=n), 2)
}

df = pd.DataFrame(data)

# Step 2: Basic exploration
print("=== ExamGuard Training Data Analysis ===\n")
print(f"Total clips: {len(df)}")
print(f"Columns: {list(df.columns)}")
print(f"\nFirst 5 rows:")
print(df.head())

# Step 3: Class distribution (THE MOST IMPORTANT CHECK)
print("\n=== Class Distribution ===")
print(df["label"].value_counts())
print(f"\nNormal: {(df['label'] == 'normal').mean():.1%}")
print(f"Cheating: {(df['label'] != 'normal').mean():.1%}")
print("WARNING: Data is very imbalanced!" if (df["label"] == "normal").mean() > 0.8 else "OK")

# Step 4: Analysis by camera
print("\n=== Detections by Camera ===")
suspicious = df[df["label"] != "normal"]
```

```
print(suspicious.groupby("camera_id")["clip_id"].count())

# Step 5: Confidence analysis
print("\n=== Confidence by Label ===")
print(df.groupby("label")["confidence"].agg(["mean", "min", "max"]))

# Step 6: Recommend training split
print("\n=== Recommended Split ===")
total_cheating = len(df[df["label"] != "normal"])
print(f"Total cheating clips: {total_cheating}")
print(f"Recommended: Oversample cheating clips to at least {len(df)//5} for balance")
```

Key Pandas Functions for ExamGuard

```
# Loading data
pd.read_csv("file.csv")        # Load CSV
df.to_csv("file.csv")          # Save CSV

# Exploring
df.head()                      # First rows
df.shape                       # (rows, columns)
df.info()                      # Column types
df.describe()                  # Statistics
df["column"].value_counts()    # Count unique values

# Filtering
df[df["label"] == "phone"]     # Filter rows
df[df["confidence"] > 0.8]     # Condition filter
df[(cond1) & (cond2)]          # Multiple conditions

# Grouping
df.groupby("column").mean()    # Group and average
df.groupby("column").count()   # Group and count

# Cleaning
df.isnull().sum()              # Check missing values
df.dropna()                    # Drop missing rows
df.fillna(value)               # Fill missing values

# Creating
df["new_col"] = values         # Add column
df["binary"] = (df["label"] != "normal").astype(int)  # Binary encoding
```

The Pandas Workflow for ExamGuard

```
1. LOAD:    pd.read_csv("exam_clips.csv")
2. EXPLORE: df.shape, df.describe(), value_counts()
3. CLEAN:   Handle missing data, fix errors
4. ANALYZE: groupby, filter, statistics
5. PREPARE: Create labels, split data
6. SAVE:    train.to_csv(), test.to_csv()
7. TRAIN:   Feed clean data to your models
```

This workflow happens BEFORE any model training. Skipping it leads to garbage models.

Matplotlib - Charts and Graphs in Python

What is Matplotlib?

Matplotlib is a Python library for creating charts, graphs, and visualizations. It turns numbers into pictures so you can understand data at a glance.

```
import matplotlib.pyplot as plt

# That's it - you can now make any chart
```

Why Matplotlib Matters for ExamGuard

You need to SEE your data and your model's performance. Numbers alone don't tell the full story.

What You Need to See	Chart Type	Why
Class distribution (normal vs cheating)	Bar chart	Check if data is imbalanced
Model accuracy over training	Line plot	See if model is learning
Loss curve during training	Line plot	Detect overfitting
Detection confidence distribution	Histogram	Are scores clustered high or spread out?
Precision vs Recall tradeoff	Line plot	Find the best confidence threshold
Alerts per hour during an exam	Bar chart	Understand system behavior
Confusion matrix	Heatmap	See what the model gets right and wrong

Without visualization, you're training blind. Every ML engineer uses Matplotlib constantly.

What to Learn

1. Line Plots (Training Curves)

The most common chart in ML: plotting how your model improves over training.

```
import matplotlib.pyplot as plt

# Simulated training data
epochs = list(range(1, 101))
train_accuracy = [0.50 + 0.004 * e + 0.001 * (e % 5) for e in epochs]  # Improving
val_accuracy = [0.48 + 0.003 * e + 0.002 * (e % 7) for e in epochs]    # Slightly lower

# Cap at 1.0
train_accuracy = [min(a, 0.98) for a in train_accuracy]
val_accuracy = [min(a, 0.95) for a in val_accuracy]

plt.figure(figsize=(10, 6))
plt.plot(epochs, train_accuracy, label="Training Accuracy", color="blue")
plt.plot(epochs, val_accuracy, label="Validation Accuracy", color="orange")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("ExamGuard Phone Detection - Training Progress")
plt.legend()
plt.grid(True)
plt.savefig("training_curve.png")
plt.show()
```

ExamGuard connection: You'll plot this for EVERY model you train. If training accuracy goes up but validation doesn't, your model is overfitting (memorizing instead of learning).

2. Bar Charts (Class Distribution)

```
import matplotlib.pyplot as plt

# ExamGuard label distribution
labels = ["Normal", "Phone", "Looking\nNeighbor", "Passing\nNotes", "Other"]
counts = [9500, 200, 180, 70, 50]
colors = ["green", "red", "orange", "red", "orange"]

plt.figure(figsize=(10, 6))
plt.bar(labels, counts, color=colors)
plt.xlabel("Behavior Label")
plt.ylabel("Number of Clips")
plt.title("ExamGuard Training Data - Class Distribution")

# Add count labels on top of bars
for i, count in enumerate(counts):
    plt.text(i, count + 100, str(count), ha="center", fontweight="bold")

plt.savefig("class_distribution.png")
plt.show()

# The visual makes the imbalance problem OBVIOUS
# 9500 normal vs 200 phone - you can SEE the problem
```

ExamGuard connection: This chart instantly shows you the imbalanced data problem. 9500 normal clips vs just 200 phone clips. You can't miss it when it's visual.

3. Histograms (Confidence Distribution)

```
import matplotlib.pyplot as plt
import numpy as np

# Simulated confidence scores
normal_confidence = np.random.normal(0.9, 0.05, 1000)    # Normal: high confidence
cheating_confidence = np.random.normal(0.65, 0.15, 100)   # Cheating: lower, more spread

plt.figure(figsize=(10, 6))
plt.hist(normal_confidence, bins=30, alpha=0.7, label="Normal", color="green")
plt.hist(cheating_confidence, bins=30, alpha=0.7, label="Cheating", color="red")
plt.xlabel("Confidence Score")
plt.ylabel("Frequency")
plt.title("ExamGuard - Detection Confidence Distribution")
plt.legend()
plt.axvline(x=0.75, color="black", linestyle="--", label="Threshold (0.75)")
plt.legend()
plt.savefig("confidence_distribution.png")
plt.show()
```

```
# This shows WHERE to set your alert threshold
# Too low = many false alarms. Too high = missed detections.
```

ExamGuard connection: Choose the confidence threshold by seeing where normal and cheating distributions overlap.

4. Scatter Plots (Feature Relationships)

```
import matplotlib.pyplot as plt
import numpy as np

# Simulated: Gaze deviation vs. Head movement frequency
np.random.seed(42)

# Normal students
normal_gaze = np.random.normal(10, 5, 200)      # Low gaze deviation
normal_movement = np.random.normal(3, 2, 200)    # Low head movement

# Cheating students
cheat_gaze = np.random.normal(35, 10, 30)       # High gaze deviation
cheat_movement = np.random.normal(15, 5, 30)     # High head movement

plt.figure(figsize=(10, 6))
plt.scatter(normal_gaze, normal_movement, c="green", alpha=0.5, label="Normal")
plt.scatter(cheat_gaze, cheat_movement, c="red", alpha=0.7, label="Cheating")
plt.xlabel("Gaze Deviation (degrees)")
plt.ylabel("Head Movements per Minute")
plt.title("ExamGuard - Student Behavior Features")
plt.legend()
plt.grid(True)
plt.savefig("behavior_scatter.png")
plt.show()

# You can see that cheating students form a clear cluster!
```

ExamGuard connection: Visualize whether your features actually separate normal from cheating behavior. If they overlap completely, those features are useless.

5. Subplots (Multiple Charts Together)

```
import matplotlib.pyplot as plt
import numpy as np

fig, axes = plt.subplots(2, 2, figsize=(14, 10))
fig.suptitle("ExamGuard - Training Dashboard", fontsize=16)

# Top left: Accuracy curve
epochs = range(1, 51)
axes[0, 0].plot(epochs, np.cumsum(np.random.normal(0.01, 0.005, 50)).clip(0.5, 0.95))
axes[0, 0].set_title("Model Accuracy")
axes[0, 0].set_xlabel("Epoch")
axes[0, 0].set_ylabel("Accuracy")

# Top right: Loss curve
```

```
axes[0, 1].plot(epochs, np.cumsum(np.random.normal(-0.02, 0.01, 50)).clip(0.1, 2.0)[::-1])
axes[0, 1].set_title("Model Loss")
axes[0, 1].set_xlabel("Epoch")
axes[0, 1].set_ylabel("Loss")

# Bottom left: Class distribution
axes[1, 0].bar(["Normal", "Phone", "Gaze", "Notes", "Other"],
           [9500, 200, 180, 70, 50], color=["green", "red", "orange", "red", "orange"])
axes[1, 0].set_title("Class Distribution")

# Bottom right: Confidence histogram
axes[1, 1].hist(np.random.normal(0.85, 0.1, 1000), bins=30, color="blue", alpha=0.7)
axes[1, 1].set_title("Confidence Scores")
axes[1, 1].set_xlabel("Confidence")

plt.tight_layout()
plt.savefig("training_dashboard.png")
plt.show()
```

ExamGuard connection: During training, you'll have a dashboard showing multiple metrics at once.

Mini Project: Plot a Model's Training History

```
"""
Mini Project: Simulate and Plot ExamGuard Model Training
Practice: Line plots, subplots, labels, legends, saving figures
"""
import matplotlib.pyplot as plt
import numpy as np

np.random.seed(42)
epochs = 100

# Simulate realistic training curves
train_acc = []
val_acc = []
train_loss = []
val_loss = []

acc = 0.50
loss = 2.0

for e in range(epochs):
    # Accuracy improves, with noise
    acc += np.random.normal(0.004, 0.002)
    acc = min(acc, 0.97)
    train_acc.append(acc)
    val_acc.append(acc - np.random.uniform(0.02, 0.05))  # Validation slightly lower

    # Loss decreases, with noise
    loss -= np.random.normal(0.015, 0.005)
    loss = max(loss, 0.08)
```

```
    train_loss.append(loss)
    val_loss.append(loss + np.random.uniform(0.01, 0.05))  # Validation slightly higher

# Create the training report
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

# Plot 1: Accuracy
ax1.plot(range(1, epochs + 1), train_acc, label="Train Accuracy", color="blue")
ax1.plot(range(1, epochs + 1), val_acc, label="Validation Accuracy", color="orange")
ax1.set_xlabel("Epoch")
ax1.set_ylabel("Accuracy")
ax1.set_title("ExamGuard Phone Detector - Accuracy")
ax1.legend()
ax1.grid(True, alpha=0.3)

# Plot 2: Loss
ax2.plot(range(1, epochs + 1), train_loss, label="Train Loss", color="blue")
ax2.plot(range(1, epochs + 1), val_loss, label="Validation Loss", color="orange")
ax2.set_xlabel("Epoch")
ax2.set_ylabel("Loss")
ax2.set_title("ExamGuard Phone Detector - Loss")
ax2.legend()
ax2.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig("examguard_training_report.png", dpi=150)
plt.show()

# Print summary
print("=== Training Summary ===")
print(f"Final Training Accuracy: {train_acc[-1]:.2%}")
print(f"Final Validation Accuracy: {val_acc[-1]:.2%}")
print(f"Final Training Loss: {train_loss[-1]:.4f}")
print(f"Final Validation Loss: {val_loss[-1]:.4f}")
print(f"Overfit gap: {train_acc[-1] - val_acc[-1]:.2%}")

if train_acc[-1] - val_acc[-1] > 0.05:
    print("WARNING: Model may be overfitting (train >> validation)")
else:
    print("Good: Training and validation are close (no overfitting)")
```

What this mini project teaches:

- Line plots - the bread and butter of ML visualization
- Subplots - showing multiple metrics side by side
- Labels, legends, titles - making charts readable
- Saving figures - export for reports
- Interpreting curves - understanding what the charts mean

Charts You'll Make for ExamGuard

BEFORE TRAINING:
- Class distribution bar chart (check imbalance)
- Feature scatter plots (check separability)
- Data quality histograms (check distributions)

DURING TRAINING:
- Accuracy curve (is the model learning?)
- Loss curve (is the loss going down?)
- Learning rate schedule (if using LR scheduling)

AFTER TRAINING:
- Confusion matrix heatmap (what does the model get right/wrong?)
- Precision-Recall curve (find optimal threshold)
- ROC curve (overall model quality)
- Sample predictions (show actual frames with detections)

Key Matplotlib Functions

```
# Basic plotting
plt.plot(x, y)           # Line plot
plt.bar(x, y)            # Bar chart
plt.scatter(x, y)        # Scatter plot
plt.hist(data, bins=30)  # Histogram

# Customization
plt.xlabel("label")      # X-axis label
plt.ylabel("label")      # Y-axis label
plt.title("title")       # Chart title
plt.legend()             # Show legend
plt.grid(True)           # Show grid

# Multiple plots
fig, axes = plt.subplots(rows, cols)

# Saving
plt.savefig("filename.png", dpi=150)

# Display
plt.show()
```

Jupyter Notebooks

What is a Jupyter Notebook?

A Jupyter Notebook is an interactive coding environment where you can write and run Python code one piece at a time, see the results immediately, and add notes alongside your code.

Think of it as a lab notebook for coding:

- Write a piece of code (a "cell")
- Run it and see the output instantly
- Write notes explaining what you did
- Move to the next piece of code

It runs in your web browser and looks like a document with runnable code blocks.

Why Jupyter Matters for ExamGuard

The ML Workflow is Experimental

Building ML models is NOT like writing a regular program where you write all the code, then run it. Instead, it's a back-and-forth process:

```
Regular Programming:
  Write all code → Run → Debug → Done

ML Development:
  Load data → look at it → try something → see result →
  adjust → try again → see result → adjust → try again →
  visualize → adjust → test → adjust → ...
```

Jupyter Notebooks are PERFECT for this because:

- You run one cell at a time (not the whole program)
- You see results immediately (charts appear inline)
- You can go back and change earlier cells
- You keep a record of everything you tried

Every ML Professional Uses Jupyter

- Google uses Colab (Google's version of Jupyter)
- Kaggle competitions use Jupyter Notebooks
- Research papers include Jupyter Notebooks
- AI courses use Jupyter for exercises

All your ExamGuard experimentation will happen in Jupyter Notebooks.

What to Learn

1. Starting Jupyter

```
# Install (if not already installed)
pip install jupyter

# Start Jupyter Notebook
```

```
jupyter notebook

# This opens a browser window where you can create notebooks
```

Or use Google Colab (free, no installation, runs in browser):

- Go to colab.research.google.com
- Create a new notebook
- Start coding (free GPU included!)

2. Understanding Cells

A notebook is made of cells. There are two main types:

Code Cells - Write and run Python code:

```
# This is a code cell
import numpy as np

data = np.random.rand(5)
print(data)
# Output appears right below the cell!
```

Markdown Cells - Write formatted notes:

```
# This is a Heading
This is a note explaining what the code does.

## Why this step matters
Because we need to understand the data before training.
```

3. Running Cells

Action	Shortcut
Run current cell	`Shift + Enter`
Run cell, stay on it	`Ctrl + Enter`
Run cell, insert new below	`Alt + Enter`
Add cell above	`A` (in command mode)
Add cell below	`B` (in command mode)
Delete cell	`DD` (press D twice in command mode)
Switch to Markdown	`M` (in command mode)
Switch to Code	`Y` (in command mode)

4. A Typical ExamGuard Notebook

Here's what a real ExamGuard experimentation notebook looks like:

```
Cell 1 (Markdown):
  # ExamGuard Phone Detection - Experiment 1
  Testing YOLOv8-nano on exam hall images

Cell 2 (Code):
  import numpy as np
  import pandas as pd
  import matplotlib.pyplot as plt

Cell 3 (Markdown):
  ## Load Training Data
```

```
Cell 4 (Code):
  df = pd.read_csv("exam_clips.csv")
  print(f"Total clips: {len(df)}")
  df.head()

Cell 5 (Markdown):
  ## Check Class Distribution

Cell 6 (Code):
  df["label"].value_counts().plot(kind="bar")
  plt.title("Class Distribution")
  plt.show()

Cell 7 (Markdown):
  ## Observation
  Data is very imbalanced: 95% normal, 5% cheating.
  Need to apply oversampling before training.

Cell 8 (Code):
  # Train the model...
```

Notice the pattern: Code → Result → Note → Code → Result → Note. This creates a complete record of your experiment.

5. Inline Visualizations

One of Jupyter's best features: charts appear RIGHT in the notebook.

```
# In a Jupyter cell:
import matplotlib.pyplot as plt
import numpy as np

x = np.linspace(0, 10, 100)
plt.plot(x, np.sin(x))
plt.title("Test Plot")
plt.show()

# The chart appears right below this cell!
# No separate window, no saving to file first.
```

6. Displaying DataFrames Nicely

```
import pandas as pd

# In Jupyter, just putting the variable name at the end of a cell
# shows it as a nice formatted table
df = pd.DataFrame({
  "camera": ["cam_1", "cam_2", "cam_3"],
  "detections": [45, 72, 31],
  "false_alarms": [5, 12, 3]
})

df  # Just the variable name - Jupyter shows it as a pretty table
```

7. Magic Commands

Jupyter has special commands that start with `%`:

```
# Time how long a cell takes to run
%%time
import numpy as np
big_array = np.random.rand(10000, 10000)
result = np.dot(big_array, big_array)

# Output: CPU times: user 2.3 s, total: 2.3 s
# This tells you if your code is fast enough for real-time!

# Show matplotlib plots inline (usually automatic)
%matplotlib inline

# List all variables in memory
%who

# Run a shell command
!pip install ultralytics
```

ExamGuard Notebook Organization

Here's how to organize your Jupyter notebooks for ExamGuard:

```
notebooks/
  01_data_exploration.ipynb       ← Explore and understand your dataset
  02_data_preprocessing.ipynb     ← Clean and prepare data for training
  03_yolo_phone_detection.ipynb   ← Train and test YOLO for phone detection
  04_cnn_gaze_tracking.ipynb      ← Train and test CNN for gaze direction
  05_lstm_behavior.ipynb          ← Train and test behavior classifier
  06_autoencoder_anomaly.ipynb    ← Train and test anomaly detector
  07_rl_alert_agent.ipynb         ← Train and test the alert decision agent
  08_integration_test.ipynb       ← Test all models together
```

Each notebook is a self-contained experiment that you can run, modify, and share.

Google Colab: Free GPU for Training

Google Colab is Jupyter in the cloud with free GPU access:

Why use Colab:

- Free GPU - Train models faster (NVIDIA T4 or better)
- No installation - Everything runs in your browser
- Pre-installed libraries - NumPy, Pandas, TensorFlow, PyTorch already there
- Google Drive integration - Save and load files from your Drive
- Share easily - Send a link to anyone

Getting started with Colab:

- 1. Go to colab.research.google.com
- 2. Click "New Notebook"
- 3. Change runtime to GPU: Runtime menu, Change runtime type, GPU

- 4. Start coding

```
# First cell in Colab - check GPU availability
import torch
print(f"GPU available: {torch.cuda.is_available()}")
if torch.cuda.is_available():
    print(f"GPU name: {torch.cuda.get_device_name(0)}")
```

ExamGuard connection: You'll train your models on Colab's free GPU. Training that takes 2 hours on CPU might take 15 minutes on GPU.

Mini Project: Your First ExamGuard Notebook

Create a new Jupyter notebook (or Colab notebook) and add these cells:

```
Cell 1 (Markdown):
    # My First ExamGuard Notebook
    Learning to use Jupyter for ML experiments

Cell 2 (Code):
    # Import libraries
    import numpy as np
    import pandas as pd
    import matplotlib.pyplot as plt
    print("All libraries loaded!")

Cell 3 (Markdown):
    ## Simulate Camera Data

Cell 4 (Code):
    # Create fake camera frame data
    frame = np.random.randint(0, 255, (100, 100, 3), dtype=np.uint8)
    print(f"Frame shape: {frame.shape}")
    print(f"Frame dtype: {frame.dtype}")

    # Display the random "image"
    plt.imshow(frame)
    plt.title("Simulated Camera Frame")
    plt.axis("off")
    plt.show()

Cell 5 (Markdown):
    ## Simulate Detection Scores

Cell 6 (Code):
    # Simulate confidence scores for 50 detections
    scores = np.random.uniform(0.2, 1.0, 50)

    plt.hist(scores, bins=15, color="steelblue", edgecolor="black")
    plt.axvline(x=0.75, color="red", linestyle="--", label="Threshold")
    plt.xlabel("Confidence Score")
    plt.ylabel("Count")
    plt.title("Detection Confidence Distribution")
```

```
plt.legend()
plt.show()

alerts = scores[scores >= 0.75]
print(f"Total detections: {len(scores)}")
print(f"Alerts (above threshold): {len(alerts)}")
print(f"Alert rate: {len(alerts)/len(scores):.1%}")
```

```
Cell 7 (Markdown):
## Observations
- The random scores are roughly uniform
- About 30% of detections exceed the 0.75 threshold
- In a real system, normal behavior would have high confidence
  and cheating would have variable confidence
```

```
Cell 8 (Code):
print("Notebook complete!")
```

What this teaches:

- How to create and run cells
- How to mix code and notes
- How to display charts inline
- The workflow of experiment, observe, note
- The structure of an ML notebook

Tips for Good Notebooks

- 1. Always add markdown cells explaining WHAT you're doing and WHY
- 2. One logical step per cell (don't cram everything into one cell)
- 3. Show your outputs - print shapes, show charts, display tables
- 4. Run cells in order - from top to bottom, don't skip around
- 5. Restart and run all before sharing (make sure it works from scratch)
- 6. Name your notebooks clearly - `03_yolo_phone_detection.ipynb` not `Untitled7.ipynb`

Phase 2: Python and Libraries - Resources

Python Basics

YouTube (Free)

- "Python Tutorial for Beginners - Full Course" by Programming with Mosh
- 6 hours, covers everything from installation to advanced topics
- Search: "Mosh Python tutorial beginners"
- "Python for Beginners" by CodeBasics
- Playlist of short videos (10-20 min each)
- Great for learning one topic at a time
- Search: "CodeBasics Python playlist"
- "Python Tutorials" by Corey Schafer
- Excellent in-depth explanations
- Covers basics through advanced topics
- Search: "Corey Schafer Python tutorial"
- "Python Tutorial for Absolute Beginners" by CS Dojo
- Very beginner-friendly
- Search: "CS Dojo Python beginners"

Courses (Free and Paid)

- "Python for Data Science" on Coursera (University of Michigan)
- Specifically focused on Python for data/ML work
- Free to audit
- "Python for Everybody" by Dr. Chuck (University of Michigan)
- Completely free on py4e.com
- Very beginner-friendly, step by step

Practice Platforms (Free)

- Kaggle Learn: Python (kaggle.com/learn/python)
- Free, interactive, hands-on exercises
- Specifically designed for data science
- HackerRank Python (hackerrank.com/domains/python)
- Practice problems from easy to hard
- Great for building confidence
- LeetCode Easy Problems (leetcode.com)
- Start with Easy Python problems
- Builds problem-solving skills

NumPy

YouTube

- "NumPy Tutorial for Beginners" by freeCodeCamp

- Complete tutorial in one video
- Search: "freeCodeCamp NumPy tutorial"
- "NumPy for Machine Learning" by Krish Naik
- Focused on ML-relevant NumPy skills
- Search: "Krish Naik NumPy ML"
- "Complete NumPy Tutorial" by Keith Galli
- Hands-on with lots of examples
- Search: "Keith Galli NumPy tutorial"

Practice

- Kaggle Learn: NumPy (integrated into their Python course)
- NumPy Official Quickstart: numpy.org/doc/stable/user/quickstart.html
- 100 NumPy Exercises: Search "100 numpy exercises github" for practice problems

Pandas

YouTube

- "Pandas Tutorial for Beginners" by Corey Schafer
- Clear, practical examples
- Search: "Corey Schafer Pandas tutorial"
- "Complete Pandas Tutorial" by Keith Galli
- Comprehensive walkthrough
- Search: "Keith Galli Pandas tutorial"
- "Pandas for Data Science" by Data School
- Focused on real data analysis tasks
- Search: "Data School Pandas tutorial"

Practice

- Kaggle Learn: Pandas (kaggle.com/learn/pandas)
- Free interactive course with exercises
- This is the BEST starting point for Pandas
- Pandas Official: 10 Minutes to Pandas
- pandas.pydata.org/docs/user_guide/10min.html
- Quick overview of essential features

Matplotlib

YouTube

- "Matplotlib Tutorial" by Corey Schafer
- Covers all chart types
- Search: "Corey Schafer Matplotlib tutorial"
- "Matplotlib Full Course" by freeCodeCamp
- Comprehensive in one video

- Search: "freeCodeCamp Matplotlib tutorial"
- "Python Data Visualization with Matplotlib" by Sentdex
- Practical, project-based
- Search: "Sentdex Matplotlib tutorial"

Practice

- Kaggle Learn: Data Visualization (kaggle.com/learn/data-visualization)
- Free course covering Matplotlib and Seaborn
- Hands-on exercises
- Matplotlib Gallery: matplotlib.org/stable/gallery/
- Hundreds of example charts with code
- Find the chart you want and copy the code

Jupyter Notebooks

YouTube

- "Jupyter Notebook Tutorial for Beginners" by Corey Schafer
- Complete walkthrough of the interface
- Search: "Corey Schafer Jupyter Notebook tutorial"
- "Google Colab Tutorial" by freeCodeCamp
- How to use free cloud notebooks with GPU
- Search: "freeCodeCamp Google Colab tutorial"

Getting Started

- Google Colab (colab.research.google.com)
- No installation needed
- Free GPU for training
- Start here if you don't want to install anything
- Jupyter Official Docs: jupyter.org
- Installation instructions
- User guide

Recommended Learning Order

Week 1-2: Python Basics
  Day 1-3: Variables, data types, if/else
  Day 4-6: Loops, lists, dictionaries
  Day 7-10: Functions
  Day 11-14: Classes, file I/O
  Practice: HackerRank Easy problems

Week 3: NumPy
  Day 1-2: Array creation and properties
  Day 3-4: Math operations and reshaping
  Day 5-7: Slicing, stacking, distance calculations
  Practice: 100 NumPy exercises (do at least 30)

Week 4: Pandas
Day 1-2: Read CSV, explore data
Day 3-4: Filtering, groupby
Day 5-7: Cleaning data, saving
Practice: Kaggle Learn Pandas course

Week 5: Matplotlib + Jupyter
Day 1-3: Line plots, bar charts, histograms
Day 4-5: Subplots, customization
Day 6-7: Create your first complete Jupyter notebook
Practice: Recreate charts from this guide

Python for ML: What You DON'T Need (Yet)

Don't get distracted by these topics. They're useful but not needed for ExamGuard right now:

- Web development (Django, Flask) - later for the dashboard
- Database programming (SQL) - later for storing alerts
- GUI programming (Tkinter) - not needed
- Advanced design patterns - not needed yet
- Decorators, generators, metaclasses - not needed yet

Focus on: Variables, loops, functions, lists, dictionaries, classes, NumPy, Pandas, Matplotlib, Jupyter. That's all you need to start building ExamGuard.

Quick Links Summary

Resource	Link	What It Covers
Kaggle Learn Python	kaggle.com/learn/python	Python basics with exercises
Kaggle Learn Pandas	kaggle.com/learn/pandas	Data manipulation
Google Colab	colab.research.google.com	Free cloud notebooks + GPU
HackerRank	hackerrank.com/domains/python	Python practice problems
py4e.com	py4e.com	Free Python course
NumPy Quickstart	numpy.org/doc/stable/user/quickstart.html	NumPy essentials
Matplotlib Gallery	matplotlib.org/stable/gallery	Chart examples with code